

WEEK-7 TASK

Unsupervised Financial Anomaly Detection via Isolation Forest from First Principles

Date: 24/09/2026

1. Aim

To implement an Isolation Forest based unsupervised anomaly detection system from first principles for financial transaction data. The system isolates unusual transactions using recursive random binary partitioning and calculates mathematical anomaly scores without relying on a pre-built Isolation Forest model.

2. Description

Students will implement an Isolation Forest anomaly detection algorithm from scratch. The implementation must contain the recursive binary partitioning logic used to construct isolation trees, an ensemble of multiple isolation trees, and the mathematical calculation of anomaly scores. The method is suitable for high-dimensional financial attributes such as transaction amount, transaction frequency, account activity, and other numerical transaction features.

3. Introduction

Financial datasets can contain unusual transactions caused by fraud, account compromise, unusual spending patterns, or data errors. In many real-world cases, labelled examples of fraud are limited. Unsupervised anomaly detection is therefore useful because it can identify observations that behave differently from the majority of the data.

Isolation Forest is based on a simple idea: anomalies are usually easier to isolate than normal observations. A random tree repeatedly selects a feature and a random split value. An unusual transaction often becomes separated from the rest of the data in only a few splits. Normal observations generally require more splits.

4. Working Principle

Step 1: Select a random subset of financial transactions. **Step 2:** Select a feature randomly. **Step 3:** Select a random split value between the minimum and maximum value of that feature. **Step 4:** Divide the data into left and right partitions. **Step 5:** Continue recursively until a stopping condition is reached. **Step 6:** Build many such isolation trees. **Step 7:** Calculate the average path length for each transaction. **Step 8:** Convert the average path length into an anomaly score.

5. Isolation Tree

An isolation tree is a binary tree. Each internal node contains a randomly selected feature and split value. Transactions smaller than or equal to the split go to the left child, while transactions greater than the split go to the right child. A node becomes external when it contains one transaction, reaches the maximum tree height, or cannot be partitioned further.

6. Mathematical Anomaly Score

Let $h(x)$ be the path length of transaction x in an isolation tree. For an ensemble of trees, the average path length is:

$$E[h(x)] = (1 / T) * \sum(h_i(x)), \quad i = 1 \dots T$$

where T is the number of isolation trees. The anomaly score can be calculated as:

$$s(x, n) = 2^{-\left(\frac{E[h(x)]}{c(n)} \right)}$$

Here, n is the sample size and $c(n)$ is the average path length of an unsuccessful search in a binary search tree. A common approximation is:

$$c(n) = 2H(n-1) - 2(n-1)/n$$

$H(k)$ is the harmonic number. A score closer to 1 indicates that the observation is easier to isolate and is therefore more anomalous. Scores closer to 0 generally indicate observations that require longer paths.

7. Algorithm / Pseudocode

```
ISOLATION_TREE(data, current_height):
    if data contains one observation:
        return external node

    if current_height >= maximum_height:
        return external node

    choose a random feature
    choose a random split between feature minimum and maximum

    left_data = observations <= split
    right_data = observations > split

    create internal node
    node.left = ISOLATION_TREE(left_data, current_height + 1)
    node.right = ISOLATION_TREE(right_data, current_height + 1)

    return node

BUILD_FOREST(data, number_of_trees):
    forest = empty list

    repeat number_of_trees times:
        sample a subset from data
        tree = ISOLATION_TREE(sample, 0)
        add tree to forest

    return forest

ANOMALY_SCORE(x, forest):
    calculate path length of x in every tree
    average the path lengths
    convert average path length to anomaly score
    return score
```

8. Python Implementation From Scratch

```
import random
import math
import numpy as np

class Node:
    def __init__(self, feature=None, split=None, left=None, right=None,
                 size=0, external=False):
        self.feature = feature
        self.split = split
        self.left = left
        self.right = right
        self.size = size
        self.external = external

def harmonic_number(n):
    if n <= 0:
        return 0
    return sum(1.0 / i for i in range(1, n + 1))

def c_factor(n):
    if n <= 1:
        return 0
    return 2 * harmonic_number(n - 1) - (2 * (n - 1) / n)

def build_tree(X, height, max_height):
    n, d = X.shape

    if n <= 1 or height >= max_height:
        return Node(size=n, external=True)
```

```

# Choose a feature having more than one unique value
candidates = [j for j in range(d) if np.min(X[:, j]) < np.max(X[:, j])]

if not candidates:
    return Node(size=n, external=True)

feature = random.choice(candidates)
low = np.min(X[:, feature])
high = np.max(X[:, feature])

split = random.uniform(low, high)

left_mask = X[:, feature] <= split
right_mask = X[:, feature] > split

# Retry if random split created an empty side
if left_mask.sum() == 0 or right_mask.sum() == 0:
    return build_tree(X, height, max_height)

left = build_tree(X[left_mask], height + 1, max_height)
right = build_tree(X[right_mask], height + 1, max_height)

return Node(feature, split, left, right, n, False)

def path_length(x, node, current=0):
    if node.external:
        # Adjustment for a leaf containing more than one point
        if node.size <= 1:
            return current
        return current + c_factor(node.size)

    if x[node.feature] <= node.split:
        return path_length(x, node.left, current + 1)
    return path_length(x, node.right, current + 1)

def build_forest(X, n_trees=100, sample_size=64):
    forest = []
    sample_size = min(sample_size, len(X))
    max_height = math.ceil(math.log2(sample_size))

    for _ in range(n_trees):
        indices = np.random.choice(len(X), sample_size, replace=False)
        sample = X[indices]
        forest.append(build_tree(sample, 0, max_height))

    return forest

def anomaly_scores(X, forest, sample_size):
    c_n = c_factor(sample_size)
    scores = []

    for x in X:
        avg_path = np.mean([path_length(x, tree) for tree in forest])

        if c_n == 0:
            score = 0.0
        else:
            score = 2 ** (-avg_path / c_n)

        scores.append(score)

    return np.array(scores)

# Example financial transaction dataset
data = np.array([
    [1200, 2, 1],
    [1500, 3, 1],
    [1100, 2, 2],
    [1300, 3, 2],

```

```

[1250, 2, 1],
[1400, 3, 2],
[1350, 2, 1],
[1600, 4, 2],
[1180, 2, 1],
[1280, 3, 1],
[15000, 20, 10],    # unusual transaction
[1320, 2, 1]
], dtype=float)

forest = build_forest(data, n_trees=100, sample_size=8)
scores = anomaly_scores(data, forest, sample_size=8)

for i, score in enumerate(scores):
    print(f"Transaction {i+1}: anomaly score = {score:.4f}")

threshold = 0.65
print("\nPotential anomalies:")
for i, score in enumerate(scores):
    if score >= threshold:
        print(f"Transaction {i+1} -> score: {score:.4f}")

```

9. Explanation of the Code

Node: Represents an internal or external tree node. **build_tree():** Recursively creates a random isolation tree. **path_length():** Finds how many splits are required to isolate a transaction. **build_forest():** Builds multiple trees using random samples. **anomaly_scores():** Averages path lengths and applies the mathematical score formula.

10. Financial Features

Possible input attributes include transaction amount, number of transactions in a time period, account age, merchant category, transaction frequency, geographic distance, and other numerical representations. Categorical attributes should be converted into appropriate numerical representations before applying the algorithm.

11. Expected Result

The program produces an anomaly score for every transaction. Transactions that are isolated using shorter average paths receive higher anomaly scores. In the example dataset, the transaction with a very large amount and unusually high activity is expected to receive a comparatively high score and can be flagged for further investigation.

12. Advantages

1. Does not require labelled fraud data.
2. Can handle multiple numerical transaction attributes.
3. Uses random partitioning, making the method relatively simple to implement.
4. Can detect unusual observations without defining every fraud pattern in advance.
5. An ensemble of trees provides more stable results than relying on one random tree.

13. Limitations

The quality of anomaly detection depends on the selected features, sample size, number of trees, and threshold. An unusual but legitimate transaction may be flagged, while sophisticated anomalies that resemble normal activity may be difficult to detect. Categorical data also requires suitable preprocessing.

14. Conclusion

Isolation Forest provides an effective unsupervised approach for detecting unusual financial transactions. By recursively applying random binary partitions, anomalous observations tend to have shorter path

lengths. The ensemble average path length is converted into a mathematical anomaly score, allowing transactions to be ranked or flagged for further analysis. Implementing the method from first principles demonstrates the underlying logic rather than treating the model as a black box.